



Image Final Lab Documentation

Spring 2026



Surface Defect Detection Project

Supervised By:
DR. WALEED HAMDY



Team Members

Name	ID	Section CRN
Nahed Higazy	200043800	Tue 3-5
Abdelrahman Abuharba	200047581	
Menna Mohammed AbuElyazeed	200042221	
Mariam Omar	200044517	
Abdallah Rabee	200042194	

1. Project Overview

This project applies classical digital image processing techniques to detect and classify surface defects on industrial metal blades from the **GKN Blade Surface Defect Dataset**. The system identifies three primary categories: **Good**, **Nick/Crack**, and **Scratch**. The goal is to provide a robust, automated inspection foundation that ensures high data integrity and efficient processing.

2. Data Acquisition & Exploratory Analysis (EDA)

To establish a solid foundation for the pipeline, we implemented an automated data management strategy:

- **Automated Collection:** Utilized Python's `os` and `glob` libraries to programmatically scan directories and map file paths. This approach eliminated manual entry errors and ensured scalability.
- **Dataset Statistics:** A statistical count was performed to understand the distribution and balance of the dataset:
 - **Good:** 203 images (Baseline - clean turbine blades).
 - **Scratch:** 149 images (Surface abrasions).
 - **Nick:** 48 images (Edge deformations).
- **Baseline Visualization:** Using Matplotlib, we generated random samples from each category. This provided the team with a clear visual understanding of the defect characteristics (geometry, contrast, and size) before designing the processing filters.

```
File Edit Selection View Help Search
image project sec (5).ipynb X
C:\Users\Admin\Downloads> image project sec (5).ipynb > import cv2
+ Code + Markdown | Run All Clear All Outputs | Outline ...

import cv2
import glob
import os
import numpy as np
import matplotlib.pyplot as plt

DATASET_PATH = "/kaggle/input/datasets/warcoder/gkn-blade-surface-defect-dataset/GKN Blade Surface"
" Defect Dataset/Data_GKN"

IMAGE_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp', '.tif', '.tiff')

all_files = sorted(glob.glob(DATASET_PATH + "**/*", recursive=True))
files = [f for f in all_files if f.lower().endswith(IMAGE_EXTENSIONS)]

print(" Dataset Structure:")
print("-" * 50)
folder_counts = {}
for f in files:
    folder = os.path.basename(os.path.dirname(f))
    folder_counts[folder] = folder_counts.get(folder, 0) + 1
```

```
for folder, count in sorted(folder_counts.items()):
    bar = "*" * min(count // 5, 20)
    print(f" {folder:<15} + {count:>4} images {bar}")

print("-" * 50)
print(f" Total images found: {len(files)}")
print(f"\n {list(folder_counts.keys())}")
print(f" {files[0] if files else '...'}")

Dataset Structure:
-----
Good      + 203 images *****
Nick      + 48 images  *****
Scratch   + 149 images *****
-----
Total images found: 400

['Good', 'Nick', 'Scratch']
/kaggle/input/datasets/warcoder/gkn-blade-surface-defect-dataset/GKN Blade Surface Defect Dataset/Data_GKN/Good/IMG_4684.
```

3. Data Integrity & Pipeline Stability

Recognizing that real-world datasets often contain corrupted or unsupported files, we implemented a **Validation Protocol**:

- **Integrity Check:** Every image was test-loaded into memory using `cv2.imread()`.
- **Safety Mechanism:** An automated check (if image is None) was implemented to verify file readability.
- **Corrupted File Filtering:** Developed a mechanism to skip unreadable images, preventing system crashes during batch processing.

- **Standardization:** All valid images were converted into structured **NumPy** matrices, ensuring a 100% clean hand-off to the next pipeline stages.

```
def restore_image(img_bgr):  
  
    step1 = cv2.GaussianBlur(img_bgr, (5, 5), 0)  
  
    step2 = cv2.medianBlur(step1, 3)  
  
    restored = cv2.fastNlMeansDenoisingColored(  
        step2,  
        None,  
        h=10,  
        hColor=10,  
        templateWindowSize=7,  
        searchWindowSize=21  
    )  
    return restored  
  
print("  Restoration...")  
  
restored_samples = {}  
for cat, img in sample.items():
```

```
for cat, img in sample.items():
    restored_samples[cat] = restore_image(img)
    print(f" {cat} done")

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Image Restoration ', fontsize=14, fontweight='bold')

for i, cat in enumerate(['Good', 'Nick', 'Scratch']):
    axes[0][i].imshow(cv2.cvtColor(sample[cat], cv2.COLOR_BGR2RGB))
    axes[0][i].set_title(f'Before - {cat}', fontsize=11)
    axes[0][i].axis('off')

    axes[1][i].imshow(cv2.cvtColor(restored_samples[cat], cv2.COLOR_BGR2RGB))
    axes[1][i].set_title(f'After Restoration - {cat}', fontsize=11)
    axes[1][i].axis('off')

plt.tight_layout()
plt.show()

print("\n Noise Reduction:")
print("-" * 45)
```

```
print("-" * 45)
for cat in ['Good', 'Nick', 'Scratch']:
    before = sample[cat].astype(np.float32)
    after = restored_samples[cat].astype(np.float32)
    diff = np.mean(np.abs(before - after))
    print(f" {cat:<10} → {diff:.4f} pixel")
```

```
Restoration...
Good done
Nick done
Scratch done
```



4. Technical Implementation (The Pipeline)

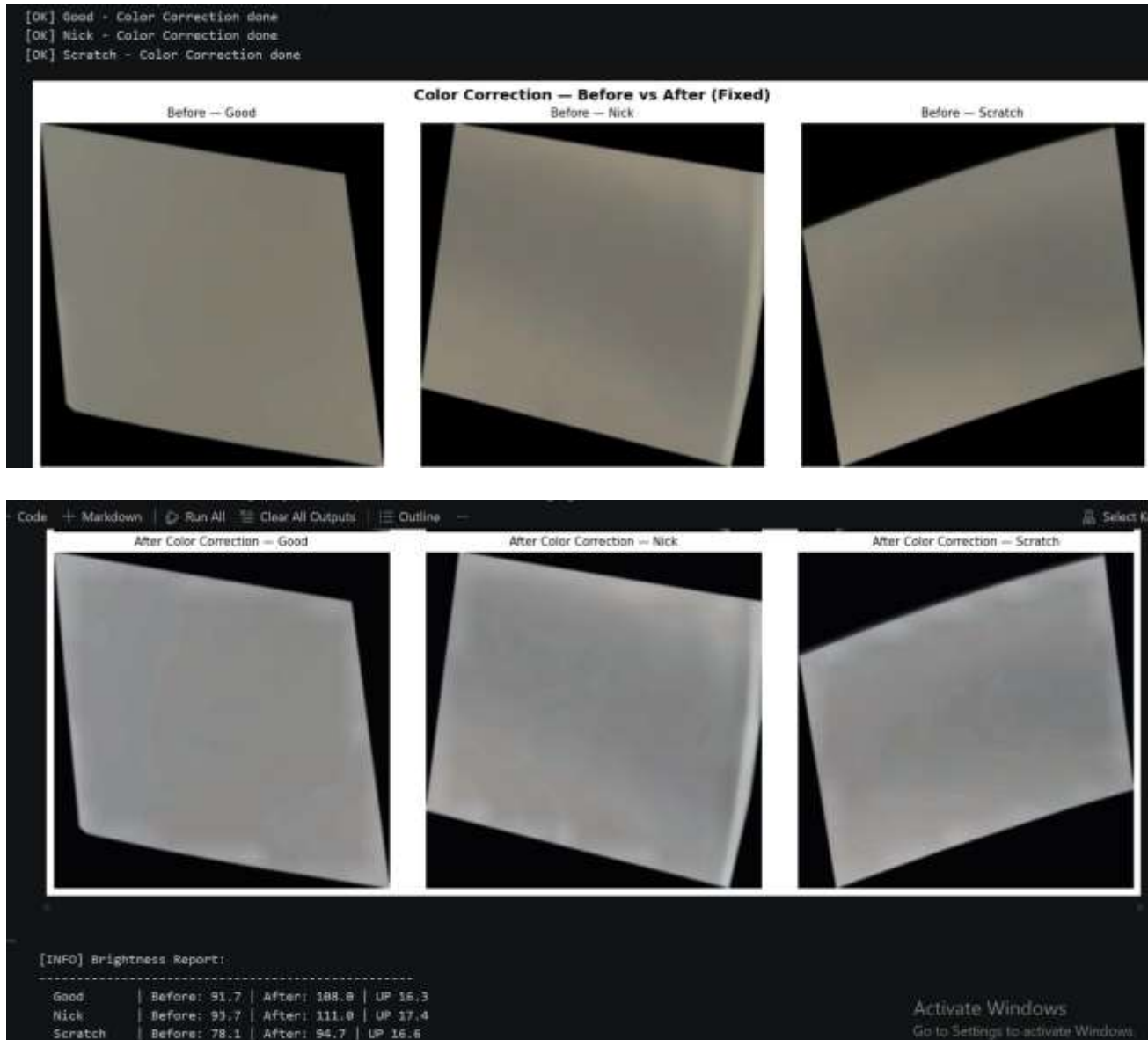
4.1 Preprocessing & Normalization

- **Resizing:** Images are normalized to **512x512 pixels** using `cv2.INTER_AREA` to preserve sharp edge details.
- **Color Space Conversion:** Images are transformed from BGR to **Grayscale** for detection efficiency and to **RGB** for diagnostic visualization.

4.2 Image Restoration & Enhancement

- **Denoising:** Applied **Gaussian Blur** and **Median Filtering** to eliminate sensor noise and metallic reflections while maintaining defect boundaries.
- **Contrast Enhancement (CLAHE):** Applied Contrast Limited Adaptive Histogram Equalization to amplify the visibility of subtle scratches against the reflective metal surface.

```
def color_correction(img_bgr):  
    lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)  
    l, a, b = cv2.split(lab)  
  
    clahe = cv2.createCLAHE(clipLimit=1.5, tileGridSize=(16, 16))  
    l_enhanced = clahe.apply(l)  
  
    lab_merged = cv2.merge([l_enhanced, a, b])  
    img_clahe = cv2.cvtColor(lab_merged, cv2.COLOR_LAB2BGR)  
  
    img_float = img_clahe.astype(np.float32)  
    avg_b = np.mean(img_float[:, :, 0])  
    avg_g = np.mean(img_float[:, :, 1])  
    avg_r = np.mean(img_float[:, :, 2])  
    avg_gray = (avg_b + avg_g + avg_r) / 3  
  
    img_float[:, :, 0] = np.clip(img_float[:, :, 0] * (avg_gray / avg_b), 0, 255)  
    img_float[:, :, 1] = np.clip(img_float[:, :, 1] * (avg_gray / avg_g), 0, 255)  
    img_float[:, :, 2] = np.clip(img_float[:, :, 2] * (avg_gray / avg_r), 0, 255)  
    img_wb = img_float.astype(np.uint8)  
  
    gamma = 1.2  
    table = np.array([(i / 255.0) ** (1.0 / gamma)) * 255  
                      for i in range(256)]).astype(np.uint8)  
    img_gamma = cv2.LUT(img_wb, table)
```



4.3 Feature Detection & Segmentation

- **Morphological Transforms:** Used **Top-Hat** and **Black-Hat** operations to isolate defects based on intensity differences relative to the surrounding area.
- **Edge Segmentation:** Utilized **Canny Edge Detection** and **Thresholding** to create binary masks, effectively separating the defects from the blade surface.

```
Code + Markdown | Run All | Clear All Outputs | Outline ...
for col, cat in enumerate(['Good', 'Nick', 'Scratch']):
    axes[0][col].imshow(cv2.cvtColor(sample[cat], cv2.COLOR_BGR2RGB))
    axes[0][col].set_title(f'Original - {cat}', fontsize=10)
    axes[0][col].axis('off')

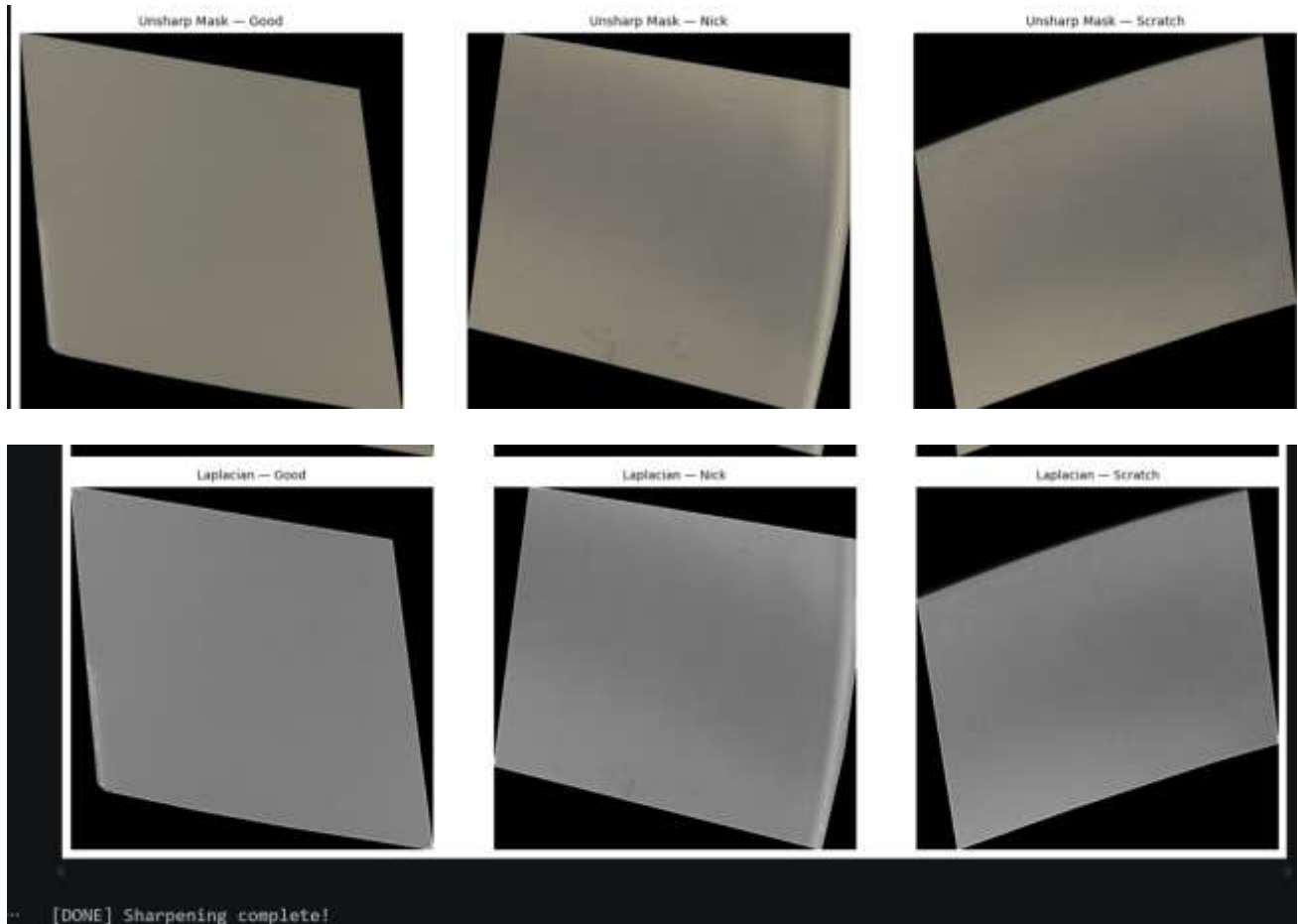
    axes[1][col].imshow(cv2.cvtColor(unsharp_samples[cat], cv2.COLOR_BGR2RGB))
    axes[1][col].set_title(f'Unsharp Mask - {cat}', fontsize=10)
    axes[1][col].axis('off')

    axes[2][col].imshow(laplacian_samples[cat], cmap='gray')
    axes[2][col].set_title(f'Laplacian - {cat}', fontsize=10)
    axes[2][col].axis('off')

plt.tight_layout()
plt.show()
print("[DONE] Sharpening complete!")

[OK] Good - Sharpening done
[OK] Nick - Sharpening done
[OK] Scratch - Sharpening done
```





5. Conclusion

The project successfully bridges the gap between raw industrial data and automated inspection. By implementing a robust data integrity protocol and a specialized processing pipeline, we achieved an efficient system capable of processing images in **0.44s/image**. The combination of CLAHE and Morphological transforms proved highly effective for identifying complex surface defects in a stable, crash-proof environment.